41039 Programming 1

Revision of Week 12: review and integration



Dr. Xuan-Bach Le
le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

Today's revision: 10 questions, about 35 minutes

#	Type	Topic
Q1	Fill in the blanks	Tracing, parsing, mutation, tests
Q2	Multiple choice	Which test reveals the bug?
Q3	Multiple choice	Where does `marks.add(mark)` go?
Q4	Predict the output	A box in a list (Java)
Q5	Predict the output	Items on sale (Python)
Q6	Find the bug	The gradebook (Java, 4 bugs)
Q7	Find the bug	Loading students (Python, 3 bugs)
Q8	Write the code	Parse one row (Java)
Q9	Write the code	Summarise a marks file (Python)
Q10	Write the code	Keep what passes a check (Java)

How it works

1. Read the question and answer it **alone** first. Write your answer down.
2. Then compare with a neighbour.
3. The answer appears on the **next click**.

The last revision: questions also use Weeks 7–11.

Week 12 in five ideas (read this first)

1. **Trace with meaning.** Say what a variable *means* after each step: `passed` = passes seen so far. A mark of exactly 50 tells >= from >.
2. **Mutation vs rebinding.** `list.add(x)` and `b.value = 7` change the *shared* object: the caller sees it. `b = new Box(9)` only moves the *local* name.
3. **A pipeline of stages.** Read, parse, validate, store, report: one job each. A number is not yet a *valid* mark.
4. **An error policy.** Say what happens to a bad row (report and skip, or stop). A rejected row never enters the accepted list; count from that list.
5. **Tests that can fail.** Empty, one item, the exact boundary, bad text, a blank line. A good test *fails* when the rule is broken.

```
read      "Mai,50"
parse     name "Mai", mark 50
validate  is 0 <= 50 <= 100?  no: report, skip
store     records.add(...)
report    count,total,passed
```

Q1 Fill in the blanks — Tracing, parsing, mutation and tests

Fill each blank with a word from the bank.

Each word is used at most once. Four words are not used.

- In `if (m >= 50) passed++;` the running count `passed` is the **(1)** .
- Turning `"85"` into 85 is **(2)** . Checking that it is in 0..100 is **(3)** .
- A test with a mark of exactly 50 sits on the **(4)** .
- In a method, `list.add(9)` is a **(5)** the caller can see. An input that makes buggy code fail is a **(6)** .

Word bank (2 min, alone)

a) validation	**f)** boundary
b) iteration	**g)** parsing
c) accumulator	**h)** constructor
d) mutation	**i)** counterexample
e) interface	**j)** inheritance

Q1 Fill in the blanks Tracing, parsing, mutation and tests

Fill each blank with a word from the bank.

Each word is used at most once. Four words are not used.

- In `if (m >= 50) passed++;` the running count `passed` is the **(1)** .
- Turning `"85"` into 85 is **(2)** . Checking that it is in 0..100 is **(3)** .
- A test with a mark of exactly 50 sits on the **(4)** .
- In a method, `list.add(9)` is a **(5)** the caller can see. An input that makes buggy code fail is a **(6)** .

Answers

(1) c) accumulator

(2) g) parsing **(3)** a) validation

(4) f) boundary **(5)** d) mutation

(6) i) counterexample

Not used: b) iteration, e) interface, h) constructor, j) inheritance.

Parse, **then** validate, **then** store. `list = new ArrayList<>()` in a method only **rebinds** the local name.

Q2 Multiple choice Which test reveals the bug?

Rule: `countPassed` returns how many marks are ≥ 50. This version has **one** bug. Which test **fails** on it, and so reveals the bug?

```
static int countPassed(int[] marks) {
    int count = 0;
    for (int i = 0; i < marks.length; i++)
        if (marks[i] > 50) count++;
    return count;
}
```

Hint: all four expected answers are right for the *rule*. Run each test through the code above. Which one gets a different number?

Choose one (90 seconds)

A. marks `{}`, expect 0
B. marks `{49, 85}`, expect 1
C. marks `{50, 70}`, expect 2
D. marks `{0, 100}`, expect 1

Q2 Multiple choice Which test reveals the bug?

Rule: `countPassed` returns how many marks are ≥ 50. This version has **one** bug. Which test **fails** on it, and so reveals the bug?

```
static int countPassed(int[] marks) {
    int count = 0;
    for (int i = 0; i < marks.length; i++)
        if (marks[i] > 50) count++;
    return count;
}
```

Choose one (90 seconds)

A. marks {}, expect 0

B. marks {49, 85}, expect 1

✓ **C.** marks {50, 70}, expect 2

D. marks {0, 100}, expect 1

Answer: C

C ✓ the code gives **1**, not 2: 50 > 50 is false, so the mark 50 is not counted. The bug: > should be >=.

A, B, D ✗ give 0, 1, 1: the right answers, *even with the bug*. None of them has a mark of exactly 50, and for every other mark > and >= agree. A test that passes with the bug is weak evidence.

Test **at** the boundary, not just near it.

Q3 Multiple choice Where does `marks.add(mark)` go?

```
String[] rows = {"Linh,85", "An,abc", "Mai,50",
                 "Tuan,120"};
ArrayList<Integer> marks = new ArrayList<>();
for (String row : rows) {
    String[] parts = row.split(",");
    try {
        int mark = Integer.parseInt(parts[1]);
        // (1)
        if (mark >= 0 && mark <= 100) {
            // (2)
        } else {
            System.out.println("Out of range");
        }
    } catch (NumberFormatException e) {
        System.out.println("Not a number");
        // (3)
    }
}
System.out.println(marks);
```

This is `main` in class `Demo`. The line `marks.add(mark);` is missing. Where must it go so the **last** line printed is `[85, 50]`?

Choose one (90 seconds)

A. At `(1)`
B. At `(2)`
C. At `(3)`
D. In the `else`, after the `println`

Q3 Multiple choice Where does `marks.add(mark)` go?

```
String[] rows = {"Linh,85", "An,abc", "Mai,50",
                 "Tuan,120"};
ArrayList<Integer> marks = new ArrayList<>();
for (String row : rows) {
    String[] parts = row.split(",");
    try {
        int mark = Integer.parseInt(parts[1]);
        // (1)
        if (mark >= 0 && mark <= 100) {
            // (2)
        } else {
            System.out.println("Out of range");
        }
    } catch (NumberFormatException e) {
        System.out.println("Not a number");
        // (3)
    }
}
System.out.println(marks);
```

This is `main` in class `Demo`. The line `marks.add(mark);` is missing. Where must it go so the **last** line printed is `[85, 50]`?

Answer: B

B ✓ only a row that was parsed **and** validated is stored.

A ✗ stores before the range check: `[85, 50, 120]`.

C ✗ does not compile: `mark` exists only inside the `try` block. `cannot find symbol ... variable mark`

D ✗ stores only the rejected mark: `[120]`.

Q4 Predict the output A box in a list

```
class Box {
    int value;
    Box(int v) { value = v; }
}
// change and main are in class Demo
static void change(Box b, ArrayList<Box> list) {
    b.value = 7;
    b = new Box(9);
    list.add(b);
}
// in main
Box x = new Box(1);
ArrayList<Box> boxes = new ArrayList<>();
boxes.add(x);
change(x, boxes);
System.out.println(x.value + " " + boxes.size());
System.out.println(boxes.get(1).value);
boxes.get(0).value++;
System.out.println(x.value);
```

Predict (90 seconds, alone)

1. Write down the **three** lines that are printed.

2. Which line of `change` affects the caller's box, and which line affects only `change`?

Q4 Predict the output A box in a list

```
class Box {
    int value;
    Box(int v) { value = v; }
}
// change and main are in class Demo
static void change(Box b, ArrayList<Box> list) {
    b.value = 7;
    b = new Box(9);
    list.add(b);
}
// in main
Box x = new Box(1);
ArrayList<Box> boxes = new ArrayList<>();
boxes.add(x);
change(x, boxes);
System.out.println(x.value + " " + boxes.size());
System.out.println(boxes.get(1).value);
boxes.get(0).value++;
System.out.println(x.value);
```

Answers

1. `7 2`
`9`
`8`

`b` starts as a copy of the reference in `x`: one box, two names. The list holds that box at index 0 and the new box (9) at index 1. `boxes.get(0)` *is* `x`'s box, so `++` makes it 8.

2. `b.value = 7` **mutates** the shared box, so `x` sees 7. `b = new Box(9)` only **rebinds** the local `b`; `x` still refers to the old box.

Q5 Predict the output Items on sale

```
class Item:
    def __init__(self, name, price):
        self.name, self.price = name, price
    def cost(self, n):
        if n <= 0:
            raise ValueError("bad n")
        return self.price * n
class Sale(Item):
    def cost(self, n):
        return super().cost(n) - 1
orders = [(Item("pen", 2), 3),
          (Sale("ink", 5), 2),
          (Item("pad", 4), 0)]
total = 0
for item, n in orders:
    try:
        total += item.cost(n)
        print(item.name, total)
    except ValueError:
        print("skip", item.name)
```

Predict (90 seconds, alone)

1. Write down the **three** lines that are printed.

2. If the ink order had `n = 0` instead of 2, what would its line be? Why?

Q5 Predict the output Items on sale

```
class Item:
    def __init__(self, name, price):
        self.name, self.price = name, price
    def cost(self, n):
        if n <= 0:
            raise ValueError("bad n")
        return self.price * n
class Sale(Item):
    def cost(self, n):
        return super().cost(n) - 1
orders = [(Item("pen", 2), 3),
          (Sale("ink", 5), 2),
          (Item("pad", 4), 0)]
total = 0
for item, n in orders:
    try:
        total += item.cost(n)
        print(item.name, total)
    except ValueError:
        print("skip", item.name)
```

Answers

1.
```
pen 6
ink 15
skip pad
```

`item.cost(n)` runs the method of the object's **own** class. For the `Sale`, that is $5 \times 2 - 1 = 9$, so the total is $6 + 9 = 15$. For `pad`, `n` is 0: `cost` raises *before* `total += ` happens, so the total stays 15.

2. `skip ink`. `super().cost(0)` raises, so `- 1` never runs. The exception passes up through `Sale.cost` to the `except`.

Q6 Find the bug The gradebook (Java)

```java
public class Gradebook {
    private int count, total, passed;
    public boolean add(int mark) {
        count++;
        if (mark < 0 || mark > 100) return false;
        total += mark;
        if (mark > 50) passed++;
        return true;
    }
    public int getCount() { return count; }
    public int getPassed() { return passed; }
    public double average() {
        double avg;
        if (count > 0) avg = total / count;
        return avg;
    }
}
```

✗ **Find 4 bugs** (3 min): 1 compile error, 3 logic errors. Give each line number and its fix.

What the class should do

- `add`: outside 0..100 → `false`, no change
- `getCount`: accepted; `getPassed`: ≥ 50
- `average()`: exact average of accepted marks; 0.0 if none

Test (`main` in class `Demo`)

```java
Gradebook g = new Gradebook();
System.out.println(g.add(50) + " "
    + g.add(-5) + " " + g.add(75));
System.out.println(g.getCount() + " "
    + g.getPassed());
System.out.println(g.average());
```

Expected output

```
true false true
2 2
62.5
```

Q6 Find the bug The gradebook (Java)

```
1  public class Gradebook {
2      private int count, total, passed;
3      public boolean add(int mark) {
4          count++;
5          if (mark < 0 || mark > 100) return false;
6          total += mark;
7          if (mark > 50) passed++;
8          return true;
9      }
10     public int getCount() { return count; }
11     public int getPassed() { return passed; }
12     public double average() {
13         double avg;
14         if (count > 0) avg = total / count;
15         return avg;
16     }
17 }
```

✗ Find 4 bugs (3 min): 1 compile error, 3 logic errors. Give each line number and its fix.

✓ The 4 fixes

- **Line 13** (compile error, reported at line 15: `variable avg might not have been initialized`): if `count` is 0, `avg` has no value. Use `double avg = 0;`
- **Line 4**: counts *before* it checks, so −5 counts. Move `count++` below line 5.
- **Line 7**: `> 50` misses exactly 50. Use `>= 50`.
- **Line 14**: `int / int` drops the `.5`: 125/2 is 62. Use `(double) total / count`.

Fix only line 13 and the test prints `3 1` and `41.0`.

Q7 Find the bug Loading students (Python)

```
1  class Student:
2      def __init__(self, name, mark):
3          self.name = name
4          self.mark = mark
5      def passed(self):
6          return self.mark > 50
7  def load(rows):
8      students = []
9      for row in rows:
10         name, text = row.split(",")
11         try:
12             mark = int(text)
13         except ValueError:
14             print("Invalid:", row)
15         students.append(Student(name, mark))
16     return students
17 rows = ["Linh,85", "An,abc", "Mai,50", "Tuan"]
18 kept = load(rows)
19 print(len(kept), sum(s.passed() for s in kept))
```

✗ **Find 3 bugs** in lines 1–16 (3 min): line numbers and fixes.

What the program should do

- A row that is not `name,<whole number>`: print `Invalid: <row>`, do **not** store it, go on
- `passed()`: `True` for a mark ≥ 50; `sum` counts the `True`s

Expected output

```
Invalid: An,abc
Invalid: Tuan
2 2
```

✗ Actual output (crashes at line 10)

```
Invalid: An,abc
ValueError: not enough values to unpack
  (expected 2, got 1)
```

Q7 Find the bug Loading students (Python)

```
1  class Student:
2      def __init__(self, name, mark):
3          self.name = name
4          self.mark = mark
5      def passed(self):
6          return self.mark > 50
7  def load(rows):
8      students = []
9      for row in rows:
10         name, text = row.split(",")
11         try:
12             mark = int(text)
13         except ValueError:
14             print("Invalid:", row)
15         students.append(Student(name, mark))
16     return students
17 rows = ["Linh,85", "An,abc", "Mai,50", "Tuan"]
18 kept = load(rows)
19 print(len(kept), sum(s.passed() for s in kept))
```

✗ Find 3 bugs in lines 1–16 (3 min): line numbers and fixes.

✓ The 3 fixes

- **Line 10**: `"Tuan"` has no comma, so the split gives one value and the unpacking raises `ValueError` *outside* the `try`: crash. Move line 10 inside the `try`.
- **Line 15**: runs even after the `except`, so a bad row is stored with the previous row's values. Move it into the `try`, after line 12.
- **Line 6**: `>` misses exactly 50 (Mai). Use `>=`.

Fix only line 10 and it prints `4 2`: the bad rows were stored.

Q8 Write the code Parse one row (Java)

Finish the `TODO` (alone, 4 min). A good row gives a `Student`; a bad row **throws**.

- `parseInt("abc")` throws `NumberFormatException`, which **is an** `IllegalArgumentException`. So do not catch it inside `parse`: the test catches it.
- One comma per row. Mark not in 0..100: `throw new IllegalArgumentException("range")`.

Skeleton

```
class Student {
    final String name;
    final int mark;
    Student(String name, int mark) {
        this.name = name;
        this.mark = mark;
    }
}
// in class Demo, next to main
static Student parse(String row) {
    // TODO
    return new Student("", 0);
}
```

Test (in `main`)

```
String[] rows = {"Linh,85", "Mai,150", "Tuan,abc"};
for (String r : rows) {
    try {
        Student s = parse(r);
        System.out.println(s.name + " " + s.mark);
    } catch (IllegalArgumentException e) {
        System.out.println("Invalid: " + r);
    }
}
```

Must print

```
Linh 85
Invalid: Mai,150
Invalid: Tuan,abc
```

Q8 Write the code Parse one row (Java)

Finish the `TODO` (alone, 4 min). A good row gives a `Student`; a bad row **throws**.

- `parseInt("abc")` throws `NumberFormatException`, which **is an** `IllegalArgumentException`. So do not catch it inside `parse`: the test catches it.
- One comma per row. Mark not in 0..100: `throw new IllegalArgumentException("range")`.

Skeleton

```
class Student {
    final String name;
    final int mark;
    Student(String name, int mark) {
        this.name = name;
        this.mark = mark;
    }
}
// in class Demo, next to main
static Student parse(String row) {
    // TODO
    return new Student("", 0);
}
```

✓ One answer (`Student` is unchanged)

```
static Student parse(String row) {
    String[] parts = row.split(",");
    int mark = Integer.parseInt(parts[1]);
    if (mark < 0 || mark > 100)
        throw new IllegalArgumentException("range");
    return new Student(parts[0], mark);
}
```

Parse, validate, then build: no `Student` with a bad mark.

Q9 Write the code Summarise a marks file (Python)

Finish the `TODO` (alone, 4 min). Each non-blank line is `name,mark`.

- Skip blank lines. A mark that is not a whole number: print `Invalid: <line>`.
- Return `(count, passed)` for the **valid** lines (passed: mark ≥ 50).

Skeleton

```
def summarise(path):
    count, passed = 0, 0
    with open(path) as f:
        for line in f:
            line = line.strip()
            # TODO
    return count, passed
```

The file's last line has no `\n`: it must still count.

Test

```
with open("marks.txt", "w") as f:
    f.write("Linh,85\n\nAn,abc\nMai,50\nChi,7.5\nBinh,49")
print(summarise("marks.txt"))
```

Must print

```
Invalid: An,abc
Invalid: Chi,7.5
(3, 2)
```

Q9 Write the code Summarise a marks file (Python)

Finish the `TODO` (alone, 4 min). Each non-blank line is `name,mark`.

- Skip blank lines. A mark that is not a whole number: print `Invalid: <line>`.
- Return `(count, passed)` for the **valid** lines (passed: mark ≥ 50).

Skeleton

```
def summarise(path):
    count, passed = 0, 0
    with open(path) as f:
        for line in f:
            line = line.strip()
            # TODO
    return count, passed
```

The file's last line has no \n: it must still count.

✓ One answer

```
def summarise(path):
    count, passed = 0, 0
    with open(path) as f:
        for line in f:
            line = line.strip()
            if line == "":
                continue
            try:
                mark = int(line.split(",")[1])
            except ValueError:
                print("Invalid:", line)
                continue
            count += 1
            if mark >= 50:
                passed += 1
    return count, passed
```

Q10 Write the code Keep what passes a check (Java)

Finish the `TODO`s (alone, 4 min) so that the test prints the output shown.

- `interface Check { boolean ok(int mark); }` is given. `InRange.ok`: `true` if `lo` $\leq$ `mark` $\leq$ `hi`.
- `keep` (`static`, in `Demo`): a **new** list of the marks where `c.ok` is `true`. Leave `marks` unchanged.

Skeleton

```
class InRange implements Check {
    private final int lo, hi;
    InRange(int lo, int hi) {
        this.lo = lo; this.hi = hi;
    }
    public boolean ok(int mark) {
        return false;  // TODO
    }
}
static ArrayList<Integer> keep(
        ArrayList<Integer> marks, Check c) {
    // TODO
    return new ArrayList<>();
}
```

Test (in `main`)

```
ArrayList<Integer> marks = new ArrayList<>(
    Arrays.asList(49, 50, 120, 85, -3));
Check valid = new InRange(0, 100);
Check pass = new InRange(50, 100);
System.out.println(keep(marks, valid));
System.out.println(keep(marks, pass));
System.out.println(marks);
```

Must print

```
[49, 50, 85]
[50, 85]
[49, 50, 120, 85, -3]
```

Q10 Write the code Keep what passes a check (Java)

Finish the `TODO`s (alone, 4 min) so that the test prints the output shown.

- `interface Check { boolean ok(int mark); }` is given. `InRange.ok`: `true` if `lo` $\leq$ `mark` $\leq$ `hi`.
- `keep` (`static`, in `Demo`): a **new** list of the marks where `c.ok` is `true`. Leave `marks` unchanged.

Skeleton

```
class InRange implements Check {
    private final int lo, hi;
    InRange(int lo, int hi) {
        this.lo = lo; this.hi = hi;
    }
    public boolean ok(int mark) {
        return false;  // TODO
    }
}
static ArrayList<Integer> keep(
        ArrayList<Integer> marks, Check c) {
    // TODO
    return new ArrayList<>();
}
```

✓ One answer

```
class InRange implements Check {
    private final int lo, hi;
    InRange(int lo, int hi) {
        this.lo = lo; this.hi = hi;
    }
    public boolean ok(int mark) {
        return lo <= mark && mark <= hi;
    }
}
static ArrayList<Integer> keep(
        ArrayList<Integer> marks, Check c) {
    ArrayList<Integer> result = new ArrayList<>();
    for (int m : marks)
        if (c.ok(m)) result.add(m);
    return result;
}
```

Looking ahead: preparing for the final assessment

What to practise next

1. **Find your gaps.** Explain one example per week to a partner; re-read the weeks you cannot.
2. **Build** the Week 12 marks report in Java *and* Python.
3. **Test it** with the list on the right.
4. **Explain causes**: input, state, output. For a bug: a counterexample, the first wrong state, the fix.

Think (1 minute)

Which week would you least like to explain to a partner today? Write it down: that is your first practice task.

Current Canvas instructions govern assessment.

Looking ahead: preparing for the final assessment

What to practise next

1. **Find your gaps.** Explain one example per week to a partner; re-read the weeks you cannot.
2. **Build** the Week 12 marks report in Java *and* Python.
3. **Test it** with the list on the right.
4. **Explain causes**: input, state, output. For a bug: a counterexample, the first wrong state, the fix.

Think (1 minute)

Which week would you least like to explain to a partner today? Write it down: that is your first practice task.

Current Canvas instructions govern assessment.

A test list for any program

- no data; exactly one item
- exactly the boundary (50) and one below (49)
- bad text (`abc`, `7.5`); out of range (`-1`, `101`)
- a missing field; a blank line; a last line with no newline

Optional: beyond the core

A filter loop like Q10's, as a Java 8 stream:

```
List<Integer> pass = marks.stream()
    .filter(m -> m >= 50 && m <= 100)
    .collect(Collectors.toList());
```

Python: `[m for m in marks if 50 <= m <= 100]`

Both give `[50, 85]` and leave `marks` unchanged.

You should now be able to...

Trace, explain and test

- ✓ say what each variable means after every step (Q1, Q6)
- ✓ choose a test that can fail: exactly 50, bad text, a blank line (Q2, Q6, Q9)
- ✓ tell mutation from rebinding (Q1, Q4)
- ✓ follow a call to the object's own method, through `super` and exceptions (Q5, Q10)

Build a small pipeline

- ✓ parse, then validate, then store, in that order (Q3, Q7, Q8)
- ✓ keep rejected rows out, and count from what you kept (Q3, Q6, Q7, Q9)
- ✓ read a text file line by line, safely (Q9)
- ✓ throw and catch the right exception (Q5, Q7, Q8)

Next: the final assessment

Current Canvas instructions govern assessment. Practise the integrated marks report in both languages, and test it with the list from Looking ahead.